



EDUCAÇÃO A DISTÂNCIA

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Unidade 2 | Semana 4

Instruções, Registradores e Ciclo de Instrução



INSTITUTO FEDERAL
Sudeste de Minas Gerais

Educação a Distância
Pró-reitoria de Ensino

Curso Técnico em Desenvolvimento de Sistemas

Professor: Jeronimo Costa Penha

Apostila V1 | 2026



Apresentação

Na Semana 3, observamos como a ULA realiza operações binárias e produz *flags*. Agora acompanharemos o passo seguinte: como uma instrução indica a operação, seleciona operandos e movimenta dados entre registradores, ULA e memória.

O ciclo de instrução conecta o programa armazenado ao funcionamento interno da CPU. Ao compreender busca, decodificação e execução, o estudante passa a interpretar uma sequência de máquina como uma série organizada de transferências e operações.

Ideia central: Cada instrução altera uma parte bem definida do estado do processador. O programa surge da sequência dessas alterações.

Objetivos de aprendizagem

- Explicar a relação entre ISA, linguagem de montagem e linguagem de máquina.
- Identificar *opcode*, operandos e campos de uma instrução.
- Descrever as funções do PC, IR, registradores gerais e registrador de estado.
- Acompanhar busca, decodificação e execução.
- Calcular endereços efetivos em modos de endereçamento básicos.
- Rastrear uma sequência com carga, operação, armazenamento e desvio.

Roteiro de estudos

1. Assista ao primeiro vídeo e acompanhe as seções 1 a 6.
2. Refaça as microoperações da busca usando valores próprios para o PC.
3. Assista ao segundo vídeo e estude as seções 7 a 13.
4. Resolva o rastreamento guiado antes de consultar a conferência.
5. Realize o questionário formativo mais de uma vez.
6. Revise as Semanas 1 a 3 antes da Avaliação 1.

Distribuição sugerida do tempo

Atividade	Tempo
Leitura da apostila	1h35
Duas videoaulas	25 min
Exemplos e rastreamento	45 min
Questionário formativo	35 min
Revisão para a Avaliação 1	25 min
Avaliação 1	1h00

1. Do programa à instrução de máquina

Linguagens de alto nível permitem expressar algoritmos com variáveis, estruturas de decisão e funções. O processador executa padrões binários definidos por sua arquitetura. Em um fluxo compilado, ferramentas traduzem o código até chegar às instruções de máquina (Figura 1).

Do programa às instruções executadas

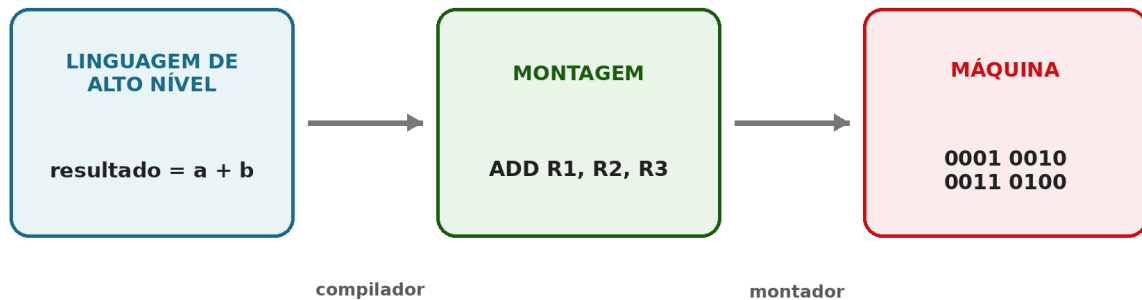


Figura 1 — Relação introdutória entre código de alto nível, linguagem de montagem e código de máquina.

A linguagem de montagem usa mnemônicos, como ADD, LOAD e STORE. O montador transforma cada instrução em uma codificação binária. Uma mesma operação pode possuir codificações diferentes em arquiteturas diferentes.

Arquitetura do conjunto de instruções

A ISA define a interface entre software e hardware. Ela especifica instruções, registradores visíveis, formatos, tipos de operandos, modos de endereçamento e regras observáveis de execução. A organização interna determina como o processador implementa essa interface.

Distinção importante: A ISA descreve o que o processador oferece ao programador. O caminho de dados e a unidade de controle mostram como uma implementação realiza esse comportamento.

2. Anatomia de uma instrução

Uma instrução precisa indicar a ação e os dados envolvidos. O *opcode* seleciona a operação. Outros campos indicam registradores, constantes, deslocamentos ou informações usadas para localizar operandos (Figura 2).

Formato didático de uma instrução de 16 bits



Exemplo hipotético: cada ISA define seus próprios campos e significados.

Figura 2 — Formato didático de 16 bits usado somente para explicar a função dos campos.

Campo	Função	Exemplo
Opcode	Seleciona a operação	ADD, LOAD, BEQ
Destino	Recebe o resultado	R1
Origem	Fornece um operando	R2
Imediato	Constante dentro da instrução	5
Deslocamento	Participa de endereço ou desvio	12

Capacidade dos campos

Um campo com k bits possui 2^k padrões. Quatro bits de *opcode* podem codificar até 16 padrões. Cinco bits para identificar registradores permitem selecionar até 32 registradores. Reservar mais bits para um campo deixa menos espaço para outros campos quando a instrução possui tamanho fixo.

Cuidado: O exemplo de 16 bits desta apostila é pedagógico. Não use seus *opcodes* como se fossem códigos de uma arquitetura real.

3. Registradores que participam do ciclo

Registrador	Papel no modelo introdutório
PC	Endereço da próxima instrução a buscar
IR	Bits da instrução atual
Uso geral	Operandos, endereços e resultados temporários
Estado	Flags como Z, N, C e V
MAR	Endereço de um acesso à memória
MDR	Dado transferido entre memória e processador

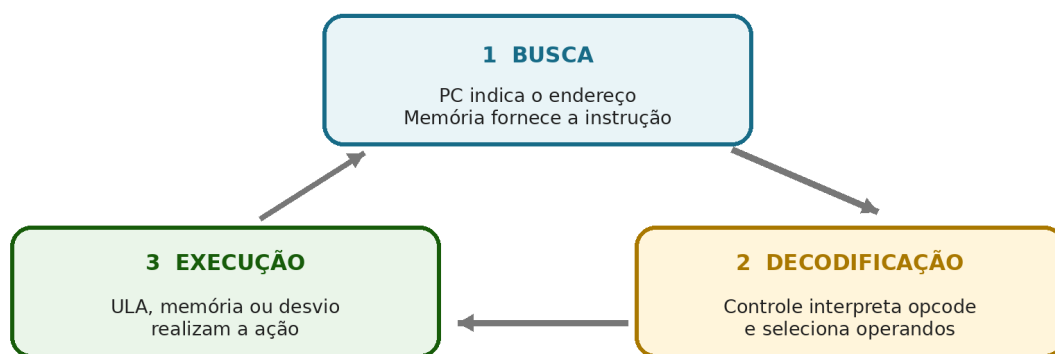
MAR e MDR ajudam a explicar transferências, mas seus nomes e sua presença explícita dependem do modelo e da arquitetura. O princípio permanece: um acesso precisa selecionar um endereço e transferir um conteúdo.

Estado arquitetural

O estado visível de uma execução inclui valores de registradores, *flags* e memória. Cada instrução lê parte desse estado e altera destinos específicos. Rastrear um programa significa registrar essas mudanças na ordem correta.

4. Ciclo de busca, decodificação e execução

Ciclo básico de uma instrução



O próximo ciclo usa o PC atualizado pela sequência normal ou por um desvio

Figura 3 — Etapas recorrentes do ciclo básico de instrução.

Busca

O PC fornece o endereço. A memória devolve a instrução e o IR a mantém durante a etapa seguinte. O PC normalmente avança para o endereço sequencial.

Decodificação

A unidade de controle interpreta o *opcode* e os campos. Ela seleciona registradores, define a operação da ULA, reconhece acessos à memória e prepara a forma de atualizar o PC.

Execução

A ação pode ser uma operação aritmética, uma transferência, uma leitura, uma escrita ou um desvio. O resultado atualiza o destino e, quando previsto, as *flags*.

Modelo: As três etapas são uma visão conceitual. Processadores reais podem subdividi-las, sobrepor-las ou executá-las por circuitos mais complexos (Figura 3).

5. Microoperações da busca

Microoperações descrevem transferências internas elementares. Em um modelo com PC, MAR, MDR e IR, a busca pode ser escrita como:

```
MAR ← PC
MDR ← Memória[MAR]
IR ← MDR
PC ← PC + tamanho da instrução
```

Exemplo resolvido

O PC contém 120 e cada instrução ocupa 4 bytes.

Passo	Microoperação	Resultado
1	MAR ← PC	MAR recebe 120
2	MDR ← Memória[MAR]	MDR recebe Memória[120]
3	IR ← MDR	IR recebe a instrução
4	PC ← PC + 4	PC recebe 124

Se a instrução em 120 for um desvio tomado, sua execução poderá substituir 124 pelo endereço de destino. A atualização sequencial continua correta como parte da busca; o desvio altera o PC depois.

6. Unidade de controle e caminho de dados

Caminho simplificado de dados e controle

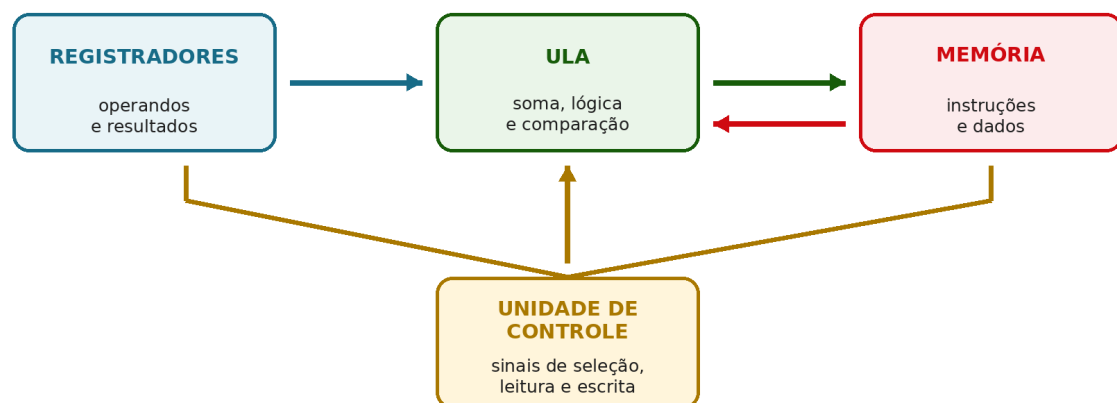


Figura 4 — Relação simplificada entre registradores, ULA, memória e unidade de controle.

A unidade de controle gera sinais. Para ADD R1, R2, R3, ela seleciona R2 e R3 como origens, escolhe soma na ULA e habilita a escrita em R1. Para STORE, seleciona o registrador que contém o dado, determina o endereço e habilita a escrita na memória (Figura 4).

Exemplo de sinais conceituais

Decisão de controle	ADD	LOAD	STORE
Operação da ULA	soma	calcular endereço	calcular endereço
Leitura de memória	não	sim	não
Escrita de memória	não	não	sim
Escrita em registrador	sim	sim	não

Interpretação: A tabela mostra decisões lógicas, não nomes universais de fios de controle.

7. Classes de instruções

Classe	Finalidade	Exemplos
Transferência	Mover dados entre registradores e memória	LOAD, STORE, MOV
Aritmética	Calcular valores numéricos	ADD, SUB, INC
Lógica	Combinar ou testar bits	AND, OR, XOR
Deslocamento	Reposicionar bits	SHL, SHR
Comparação	Produzir condições e flags	CMP
Controle	Alterar a sequência de execução	BEQ, JUMP, CALL

Uma expressão simples de alto nível pode exigir várias instruções. Se os valores estiverem na memória, o processador pode carregá-los, realizar a operação em registradores e armazenar o resultado.

Exemplo conceitual para resultado = a + b

```
LOAD R2, [endereço_de_a]
LOAD R3, [endereço_de_b]
ADD R1, R2, R3
STORE R1, [endereço_do_resultado]
```

8. Modos de endereçamento

Cinco maneiras de localizar o operando

IMEDIATO	ADDI R1, R2, 5	o valor 5 está na instrução
REGISTRADOR	ADD R1, R2, R3	R2 e R3 contêm os operandos
DIRETO	LOAD R1, [200]	a instrução informa o endereço 200
INDIRETO	LOAD R1, [R4]	R4 contém o endereço
BASE + DESLOCAMENTO	LOAD R1, 12(R4)	endereço efetivo = $R4 + 12$

Figura 5 — Formas introdutórias de obter operandos ou endereços.

O modo imediato evita outro acesso para buscar a constante, mas seu intervalo depende do tamanho do campo. O modo por registrador usa dados já próximos da ULA. Os modos direto, indireto e base mais deslocamento permitem localizar dados em memória (Figura 5).

Endereço efetivo: É o endereço realmente enviado ao sistema de memória. Ele não é o conteúdo encontrado nesse endereço.

9. Cálculo do endereço efetivo

Exemplo 1 — direto

```
LOAD R1, [200]
endereço efetivo = 200
```

Exemplo 2 — indireto por registrador

```
R4 = 200
LOAD R1, [R4]
endereço efetivo = 200
```

Exemplo 3 — base mais deslocamento

```
R4 = 1000
LOAD R1, 12(R4)
endereço efetivo =  $1000 + 12 = 1012$ 
```

Se Memória[1012] contém 37, R1 recebe 37. O valor 1012 localiza o dado; não é necessariamente o dado. R4 permanece com 1000, salvo quando a instrução especifica atualização automática.

Exemplo 4 — deslocamento negativo

```
R6 = 500
LOAD R2, -20(R6)
endereço efetivo = 500 - 20 = 480
```

10. Rastreamento de uma sequência

Considere R1 = 0, R2 = 7, R3 = 5 e Memória[300] = 4.

```
LOAD R4, [300]
ADD R1, R2, R3
SUB R1, R1, R4
STORE R1, [304]
```

Passo	Instrução	Leitura	Alteração
1	LOAD R4, [300]	Memória[300] = 4	R4 = 4
2	ADD R1, R2, R3	R2 = 7; R3 = 5	R1 = 12
3	SUB R1, R1, R4	R1 = 12; R4 = 4	R1 = 8
4	STORE R1, [304]	R1 = 8	Memória[304] = 8

A coluna “alteração” registra somente o destino. R2, R3 e Memória[300] não mudam. Esse cuidado impede que um valor lido seja confundido com um valor sobrescrito.

11. Desvios e flags

Desvio condicional e atualização do PC

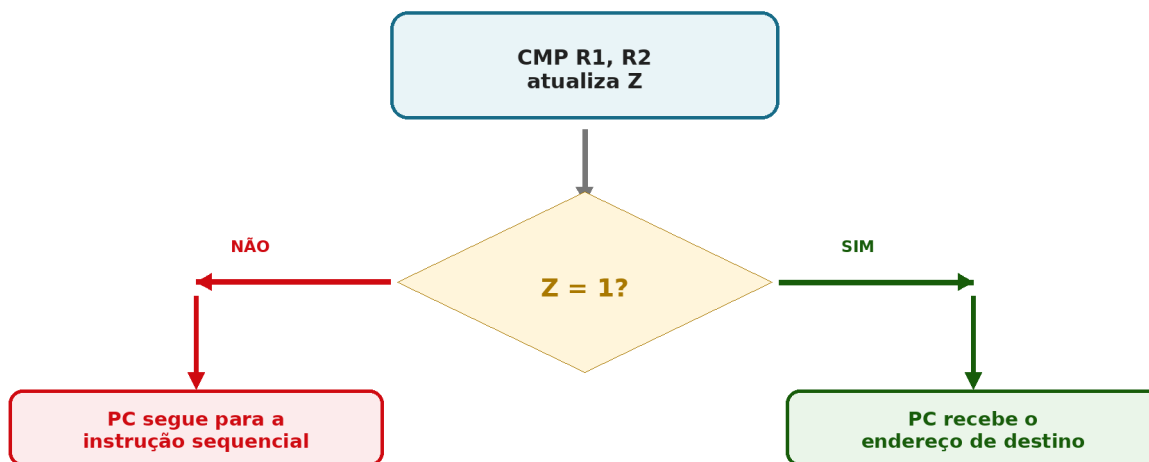


Figura 6 — Decisão de um desvio de igualdade baseada na flag Z.

CMP R1, R2
BEQ destino

CMP pode calcular internamente $R1 - R2$ e atualizar *flags*. Se os valores forem iguais, a diferença será zero e Z receberá 1. BEQ consulta essa condição: com $Z = 1$, o PC recebe o destino; com $Z = 0$, a execução segue para a próxima instrução (Figura 6).

Exemplo resolvido

```
400: CMP  R1, R2
404: BEQ  420
408: ADD  R3, R3, R4
420: STORE R1, [500]
```

Se $R1 = 9$ e $R2 = 9$, CMP ativa Z. Após buscar BEQ em 404, o PC teria avançado para 408, mas o desvio tomado substitui esse valor por 420. A ADD em 408 não é executada nesse caminho.

Dependência: Uma instrução pode usar uma condição produzida pela instrução anterior. Por isso, ordem e estado das *flags* importam.

12. Instruções e dados na memória

No modelo de programa armazenado, instruções e dados aparecem como bits na memória. O contexto determina a função do acesso. O PC indica uma busca de instrução; LOAD e STORE calculam endereços de dados.

Uma implementação pode possuir caminhos e caches separados para instruções e dados, mesmo oferecendo ao software um espaço de endereçamento coerente. A hierarquia de memória e as diferenças entre cache, RAM e armazenamento serão aprofundadas na Unidade 3.

Limite desta semana: Interrupções, dispositivos e detalhes de cache ficam para as próximas semanas. Aqui o foco é a execução normal de uma instrução.

13. O que varia entre arquiteturas

Aspecto	Possíveis variações
Registradores	quantidade, largura e funções especiais
Formato	tamanho fixo ou variável e posição dos campos
Operandos	quantidade e locais permitidos
Endereçamento	modos e intervalos de deslocamento
Flags	quais operações atualizam cada indicador
Controle	formas internas de implementar a ISA

Essas diferenças serão importantes quando compararmos filosofias RISC e CISC. Nesta semana, basta reconhecer que os exemplos usam uma CPU didática e que a lógica geral precisa ser adaptada à ISA estudada.

14. Estratégia para interpretar uma instrução

1. Identifique a operação indicada pelo *opcode* ou mnemônico.
2. Localize os operandos de origem e o destino.
3. Determine se algum operando é imediato ou vem da memória.
4. Calcule o endereço efetivo quando necessário.
5. Execute a operação respeitando largura e interpretação.
6. Atualize somente o destino e as *flags* previstas.
7. Determine o próximo valor do PC.
8. Registre o novo estado antes de passar à instrução seguinte.

Regra prática: Em um rastreamento, não avance enquanto origem, destino, endereço efetivo e próximo PC não estiverem explícitos.

15. Exploração guiada

Estado inicial: R1 = 3, R2 = 8, R5 = 100, Memória[104] = 6 e Memória[108] = 0.

```
LOAD R3, 4(R5)
ADD R4, R2, R3
SUB R4, R4, R1
STORE R4, 8(R5)
CMP R4, R2
BEQ destino
```

Passo	Endereço efetivo	Destino	Novo valor
LOAD	_____	R3	_____
ADD	não se aplica	R4	_____
SUB	não se aplica	R4	_____
STORE	_____	Memória[____]	_____
CMP	não se aplica	Z	_____
BEQ	destino ou sequência	PC	_____

Conferência comentada

LOAD usa $100 + 4 = 104$ e R3 recebe 6. ADD produz $R4 = 8 + 6 = 14$. SUB produz $R4 = 14 - 3 = 11$. STORE usa $100 + 8 = 108$ e grava 11. CMP compara 11 com 8, portanto $Z = 0$. BEQ não toma o desvio e o PC segue para a instrução sequencial.

16. Exercícios de consolidação

Parte A — instruções e ciclo

1. Explique a diferença entre opcode e operando.
2. Descreva as funções do PC e do IR.
3. Escreva as quatro microoperações da busca do modelo estudado.
4. Se $PC = 640$ e a instrução ocupa 4 bytes, determine o PC sequencial.
5. Explique por que uma instrução de desvio pode substituir o PC já incrementado.

Parte B — endereçamento

1. Classifique `ADDI R1, R2, 9` e `ADD R1, R2, R3` quanto à origem dos operandos.
2. Com $R4 = 900$, calcule o endereço efetivo de `LOAD R1, 24(R4)`.
3. Com $R5 = 320$, calcule o endereço efetivo de `STORE R2, -16(R5)`.
4. Explique a diferença entre `LOAD R1, [400]` e `LOAD R1, [R4]`.
5. Se $R4 = 400$ e $Memória[400] = 25$, determine $R1$ após `LOAD R1, [R4]`.

Parte C — rastreamento e desvios

1. $R2 = 10$ e $R3 = 4$. Execute `ADD R1, R2, R3` e `SUB R1, R1, R3`.
2. $Memória[500] = 7$. Execute `LOAD R4, [500]` e `STORE R4, [504]`.
3. $R1 = 12$ e $R2 = 12$. Determine Z após `CMP` e o comportamento de `BEQ`.
4. $R1 = 12$ e $R2 = 9$. Determine Z após `CMP` e o comportamento de `BEQ`.
5. Explique por que a instrução logo após um desvio tomado não é executada nesse caminho.

17. Respostas comentadas

Parte A

Item	Resposta
1	Opcode identifica a operação; operandos indicam dados, origens ou destino.
2	PC aponta a próxima busca; IR mantém a instrução atual.
3	$MAR \leftarrow PC$; $MDR \leftarrow Memória[MAR]$; $IR \leftarrow MDR$; $PC \leftarrow PC + tamanho$.
4	644.
5	A busca prepara o endereço sequencial; a execução do desvio escolhe outro destino quando a condição é verdadeira.

Parte B

Item	Resposta
1	ADDI usa constante imediata 9; ADD usa R2 e R3.
2	$900 + 24 = 924$.
3	$320 - 16 = 304$.
4	No direto, 400 está na instrução; no indireto, R4 contém o endereço.
5	R1 recebe 25.

Parte C

Item	Resposta
1	R1 recebe 14 e depois 10.
2	R4 recebe 7; Memória[504] recebe 7.
3	$Z = 1$ e BEQ toma o desvio.
4	$Z = 0$ e BEQ segue sequencialmente.
5	O desvio altera o PC; a próxima busca ocorre no endereço de destino.

18. Resumo

- A ISA define as instruções que o processador oferece ao software.
- O opcode seleciona a operação e os outros campos identificam operandos.
- O PC aponta a próxima instrução e o IR mantém a instrução atual.
- Busca, decodificação e execução formam o ciclo básico.
- A unidade de controle coordena registradores, ULA, memória e PC.
- O modo de endereçamento define como localizar o operando.
- O endereço efetivo é diferente do conteúdo lido ou escrito.
- Desvios consultam condições e podem alterar o PC.

Checklist de domínio

- Consigo diferenciar ISA, montagem e código de máquina.
- Consigo identificar opcode, origem e destino.
- Consigo explicar PC, IR, MAR, MDR e flags no modelo estudado.
- Consigo acompanhar uma busca por microoperações.
- Consigo calcular um endereço efetivo.
- Consigo rastrear uma sequência e decidir se um desvio será tomado.

Glossário

Termo	Definição
ISA	Interface arquitetural que define instruções e recursos visíveis ao software
Opcode	Campo que identifica a operação
Operando	Dado, registrador, constante ou endereço usado pela instrução
PC	Registrador que indica a próxima instrução a buscar
IR	Registrador que mantém a instrução atual
Microoperação	Transferência ou ação interna elementar
Endereço efetivo	Endereço realmente usado em um acesso
Desvio	Instrução capaz de alterar a sequência por meio do PC

Referências

PATTERSON, David A.; HENNESSY, John L. Organização e Projeto de Computadores: a interface hardware/software. Rio de Janeiro: Elsevier, 2017.

TANENBAUM, Andrew S.; AUSTIN, Todd. Organização Estruturada de Computadores. São Paulo: Pearson, 2013.

WEBER, Raul Fernando. Fundamentos de Arquitetura de Computadores. Rio de Janeiro: Bookman, 2012.

Material elaborado para a disciplina Arquitetura e Organização de Computadores — IF Sudeste MG, 2026.